

01 / VERSTEHEN UND UMRECHNEN

Abschlussarbeit · Tag 4

Name: _____

Datum: _____

Umfang: 100 Punkte, etwa 120 Minuten als Orientierung, ohne Zeitdruck. Seiten 1–4 zuerst ohne Python bearbeiten; danach darfst du mit dem Cheatsheet nacharbeiten. Ergänzungen kenntlich machen. Auf Seiten 5–6 sind Python, Terminal und Cheatsheet erlaubt. Rechenwege zählen auch bei falschen Endergebnissen. Leerzeichen in Bitmustern dienen nur der Lesbarkeit. Alle signed Zahlen sind Zweierkomplementzahlen. Es gibt keine versteckten Lösungen in diesem Blatt.

1. Das Warum erklären

8 Punkte

a) (4 P) Ein Vier-Bit-Register enthält 1110. Warum reicht das Bitmuster allein nicht aus, um seine Bedeutung zu bestimmen? Nenne die Werte unsigned und signed und erkläre, welche Vereinbarung den Unterschied macht.

b) (4 P) Warum ist Zweierkomplement für eine Addierschaltung praktisch? Erkläre es an einer selbst gewählten Subtraktion mit vier Bits. Zeige die Bitmuster, die Addition und das gespeicherte Ergebnis.

2. Zahlensysteme und Einheiten

12 Punkte

a) (8 P) Ergänze die Tabelle. Alle Werte hier sind *unsigned*. Verwende genau acht Bits und zwei Hexziffern. Je Lücke 1 Punkt.

Dezimal	Binär (8 Bits)	Hex
78	_____	_____
_____	1010 1101	_____
_____	_____	0xD3
_____	_____	0xB7

b) (2 P) Ein Datenwort hat 24 Bits. Wie viele Bytes und wie viele Hexziffern sind das?

c) (2 P) Wie viele Bytes sind 1 KiB und 1 kB?

Name: _____

02 / DARSTELLUNGEN

Zweierkomplement und Bitbreite

3. Zahlenbereiche und Interpretationen

20 Punkte

a) (4 P) Gib jeweils Minimum und Maximum an. Jede Bereichsgrenze zählt 1 Punkt.

Darstellung	Minimum	Maximum
5 Bit unsigned	_____	_____
5 Bit Zweierkomplement	_____	_____

b) (6 P) Werte die Aufrufe aus: Bitmuster in der vorgegebenen Breite, Grenze $2^{\text{bits}-1}$, ausgeführter Rückgabezweig und Ergebnis. Je Aufruf 2 Punkte. Rechenweg angeben.

```
from_twos_complement(21, 5)
from_twos_complement(21, 8)
from_twos_complement(128, 8)
```

c) (4 P) Stelle -22 mit acht Bits dar. Beginne mit $+22$, invertiere, addiere 1 und kontrolliere über die signed Stellenwerte. Gib zusätzlich Hex an.

d) (3 P) Erweitere 10101 von fünf auf acht Bits, sodass sein signed Wert erhalten bleibt. Erkläre deine Ergänzung. Welcher signed Wert entstünde stattdessen durch Ergänzen von drei Nullen?

e) (3 P) Warum kann man die kleinste negative Vier-Bit-Zahl nicht als positive Zahl mit derselben signed Bitbreite darstellen? Zeige, was beim Invertieren und Addieren von 1 passiert.

03 / RECHNEN

Addition, Subtraktion und Overflow

4. Die Hardware rechnet mit acht Bits

20 Punkte

Für **a–c** gilt: Beide Operanden sind signed. Gib die mathematische Dezimalsumme, die vollständige Binärsumme, die acht gespeicherten Bits, deren signed Wert sowie Carry-out und signed Overflow an. Begründe die beiden Flags.
Je Fall 5 P: Dezimalsumme (1), Binärrechnung und gespeicherter Wert (2), beide Flags mit Begründung (2).

a) 0110 1110 + 0010 1101

b) 1111 1001 + 0000 0111

c) 1000 1000 + 1110 1100

d) (5 P) Berechne $23 - 41$ durch Addition im Acht-Bit-Zweierkomplement. Zeige: Negation des zweiten Operanden (2), Binäraddition und gespeicherten signed Wert (2), signed Overflow mit Begründung (1).

04 / PYTHON LESEN

Schritt für Schritt statt Raten

5. Texte, Operatoren und Fehlersuche

15 Punkte

a) (5 P) Was steht nach jeder Zeile in der Variablen? Schreibe *Textwerte mit Anführungszeichen*, Zahlen ohne. Je Variable 1 Punkt.

```
a = hex(13)
b = a[2:]
c = b.upper()
d = (9 + 3) // 4
e = c.zfill(d)
```

a = _____ b = _____ c = _____

d = _____ e = _____

b) (4 P) Berechne die Ausdrücke. Begründe kurz mit Rechenreihenfolge, vollständigen Gruppen oder Rest. Je Ausdruck 1 Punkt.

```
(16 + 3) // 4
16 + 3 // 4
19 % 16
-5 % 16
```

c) (3 P) Die Eingabeprüfungen seien korrekt. Finde den Fehler in folgender Dekodierung. Nenne einen Vier-Bit-Aufruf, bei dem sie falsch rechnet, das falsche und das richtige Ergebnis sowie die Korrektur.

```
grenze = 2**(bits - 1)
if raw <= grenze:
    return raw
return raw - 2**bits
```

d) (3 P) Erkläre je einen Unterschied:

1. return ergebnis und print(ergebnis);
2. die Zahl 9 und der Text "1001";
3. ValueError für ungültige Operanden und overflow = True.

05 / SELBST IMPLEMENTIEREN

Deine vier Funktionen

6. Python-Praxisteil

15 Punkte

Erstelle im Tag-4-Ordner eine neue Datei `abschluss_tag04.py`. Implementiere alle vier Funktionen dort. Du darfst dein Cheatsheet nachschlagen; schreibe und erkläre den Code selbst. Keine Imports nötig. Ganze Zahlen als Eingabetypen voraussetzen. Beliebige positive Bitbreiten unterstützen.

```
def to_bin(value: int, bits: int) -> str:
    pass

def to_hex(value: int, bits: int) -> str:
    pass

def from_twos_complement(raw: int, bits: int) -> int:
    pass

def add_with_overflow_flag(a: int, b: int,
                           bits: int) -> tuple[int, bool]:
    pass
```

Vertrag und Punkte:

- **to_bin (3 P):** unsigned Eingabe; Text ohne Präfix, genau bits Ziffern; links Nullen ergänzen; Bitbreite und Wertebereich prüfen.
- **to_hex (3 P):** unsigned Eingabe; große Hexbuchstaben ohne Präfix; aufgerundet bits / 4 Stellen; tatsächlichen Bitbereich prüfen, auch wenn bits kein Vielfaches von vier ist.
- **from_twos_complement (4 P):** unsigned Wert eines Musters entgegennehmen, signed Ganzzahl zurückgeben; gültige Eingaben prüfen; Vorzeichengrenze korrekt behandeln.
- **add_with_overflow_flag (5 P):** signed Operanden prüfen; richtige Summe bilden; signed Overflow erkennen; Bitbreite begrenzen; (gespeicherter signed Wert, bool) zurückgeben.

Für alle Funktionen: bits < 1 und Werte außerhalb des jeweiligen Eingabebereichs lösen ValueError aus. Ein Overflow der Summe bei gültigen Operanden löst dagegen keinen Fehler aus.

Erkläre zwei deiner eigenen Codezeilen

Wähle eine Zeile zur Bitbegrenzung und eine zur Interpretation oder Overflow-Erkennung. Schreibe jeweils: Was macht sie, und warum brauchen wir sie? Dies hilft bei der Beurteilung deines Praxiscodes, ohne zusätzliche Punkte.

06 / VERIFIKATION

Tests und persönliche Auswertung

7. Beweise, dass dein Code funktioniert

10 Punkte

Erstelle daneben `test_abschluss_tag04.py`. Importiere die Funktionen aus `abschluss_tag04`. Schreibe mindestens **20 ausgeführte Testfälle**; Parametrisierung ist erlaubt. Leite Erwartungen zuerst selbst her.

Abdeckung (6 P): Je 1 Punkt pro Kategorie mit passenden Erwartungen:

1. Binär/Hex: Null, führende Nullen, oberer unsigned Rand.
2. Hex: eine Bitbreite, die kein Vielfaches von vier ist.
3. Dekodierung: größter positiver Wert, kleinster negativer Wert und -1 .
4. Addition: ohne Overflow sowie positiver und negativer signed Overflow.
5. Mindestens ein gültiger Ein-Bit-Fall und ein Fall mit mehr als 64 Bits.
6. Ungültige Bitbreite und ungültige unsigned sowie signed Eingaben.

Umsetzung (4 P): Mindestens 20 Fälle (2), korrektes Prüfen von `ValueError` mit `pytest.raises` (1), vollständiger Testlauf und nachvollziehbare Dokumentation (1).

Import-Gerüst, keine fertigen Testfälle:

```
import pytest
from abschluss_tag04 import (
    to_bin, to_hex, from_twos_complement,
    add_with_overflow_flag,
)
```

Im Terminal aus dem Tag-4-Ordner:

```
source "$HOME/.venvs/ai-hardware-day04/bin/activate"
python -m pytest -q test_abschluss_tag04.py
```

Dieser Aufruf prüft gezielt deine Abschlussdatei. `assert` vergleicht Ist- und Sollwert. Für Fehlerfälle gehört der Funktionsaufruf in den Block `with pytest.raises(ValueError):`.

Testlauf: Datum _____ Bestanden _____ Fehlgeschlagen _____

Ein Fehler, den ich mit einem Test gefunden habe (oder ein gezielt abgesicherter Randfall): Eingabe, Erwartung und Begründung.

Abgabe und Rückblick

Gib das ausgefüllte Blatt bzw. deine nummerierten Antworten zusammen mit den zwei Python-Dateien ab. Markiere, wo du nachgeschlagen hast; das ist kein Fehler.

Das Warum kann ich jetzt so erklären:

Hier brauche ich noch eine Erklärung:

Punkte: 1/8 2/12 3/20 4/20 5/15 6/15 7/10 Gesamt: _____/100.

Grundlage: Lernplan Woche 1, Tag 4; DDCA RISC-V Edition, Abschnitt 1.4; unsere Python-Übung. Die Punkte dienen der Rückmeldung, nicht einer Schulnote.

Name: _____